How does Gaussian noise affect the Stringy05 index?

A simulation study using four patterns and eight sample sizes

Table of contents

1 Introduction 1

2 Simulation setup 2
- 2.1 Four patterns . . . 2
- 2.2 How noise was added . . . 2
- 2.3 Sample sizes, noise levels and repetitions . . . 2
- 2.4 How the simulations were run . . . 3

3 Load and combine simulation results 3
- 3.1 Summary of the results . . . 4

4 What do the original patterns look like? 5

5 How does each pattern change as noise increases? 7

6 How does Stringy05 change with noise? 10
- 6.1 Mean Stringy05 for the four patterns at n = 500 . . . 10
- 6.2 Variation across repetitions . . . 12

7 Does sample size change the response to noise? 13
- 7.1 Noise-response curves for every sample size . . . 14
  - 7.1.1 Noise sensitivity and variability across sample sizes . . . 14
- 7.2 Heatmap of pattern, sample size and noise . . . 16
- 7.3 Interpretation of results . . . 17

1 Introduction

This report investigates how the **rescaled Stringy05 scagnostic** changes when Gaussian noise is added to two-dimensional data. Stringy05 describes a geometric property of a scatterplot using a minimum spanning tree (MST). We want to learn how much noise the index can tolerate, whether its behaviour depends on the type of pattern, and whether results change with sample size.

The eventual goal is to design synthetic datasets with **500 observations** for experiments in projection pursuit. The simulations here help us choose noise levels based on measured changes in Stringy05 rather than visual judgement alone.

In our rescaled index, **0 indicates that Stringy05 has reached its Gaussian-reference floor**. It does not prove that there is no relationship between the variables. A positive value indicates that the index measures some Stringy structure above that floor.

2 Simulation setup

2.1 Four patterns

We study the following noise-free patterns:

- **Cubic:** `poly(t, degree = 3)[, 3]`, where `t` is drawn from a standard normal distribution.
- **Quartic:** `poly(t, degree = 4)[, 4]`, using the same type of input variable.
- **Sine:** `sin(pi * t)`, where `t` is standard normal.
- **Two moons:** two interleaving semicircular arcs.

The cubic and quartic patterns use R's *orthogonal polynomial basis*, rather than simply `t^3` and `t^4`. For every pattern, the original X and Y coordinates are standardised separately to have mean 0 and sample standard deviation 1.

2.2 How noise was added

Noise was added independently to **both X and Y**:

$$X_{i,\sigma,r} = X_{i,0} + \varepsilon_{x,i,\sigma,r}, \qquad Y_{i,\sigma,r} = Y_{i,0} + \varepsilon_{y,i,\sigma,r},$$

where both noise terms are drawn independently from $N(0, \sigma^2)$.

Scagnostics describe **bivariate scatterplots**, so perturbing both coordinates is a useful symmetric way to investigate changes in their geometry. Adding noise only to Y would instead investigate a particular directional noise model. The noisy coordinates are not standardised again after adding noise.

2.3 Sample sizes, noise levels and repetitions

Setting	Values
Patterns	Cubic, quartic, sine, two moons
Sample sizes	100, 200, 300, 500, 750, 1000, 1500, 2000
Noise SD	20 equally spaced values from 0 to 1
Repetitions	100 per pattern, sample size and noise level
Base seed	1111
Index	`cassowaryr::sc_stringy05(..., rescale = TRUE)`

There are $4 \times 8 \times 20 \times 100 = 64{,}000$ planned index calculations.

For each combination of pattern and sample size, the original structure is generated once and kept fixed. Every repetition adds a new noise realisation to this same structure. Therefore, the differences between repetitions come from noise, not from a new sample of the original curve. At noise SD = 0, all 100 repetitions have the same data and score.

The original structure uses `set.seed(1111)`. Noise repetition 1 uses seed 1112, repetition 2 uses 1113, and so on through 1211. The same repetition seed is used at each noise level, so noise realisations can be compared consistently across levels.

2.4 How the simulations were run

The simulations were run using an R script and a Bash launcher in the terminal. This example runs the quartic pattern at sample size 100, with 100 repetitions at 20 noise levels:

```
bash experiments/stringy_index/stringy_noise.sh \
  quartic \
  100 \
  1 \
  100 \
  1111 \
  experiments/stringy_index/results/quartic_n100.csv \
  "$(Rscript -e 'cat(paste(seq(0, 1, length.out = 20), collapse = ","))')"
```

Each completed Stringy05 result was appended to a CSV file. The output records the pattern, sample size, noise SD, repetition, noise seed and score. With the generating code and seed, the original structure and each noisy point cloud can be recreated without storing every noisy dataset.

3 Load and combine simulation results

All simulation CSV files are stored in `experiments/stringy_index/results/`. We read and combine them into one table for analysis.

```
library(tidyverse)
library(here)

# Main settings for this report
base_seed <- 1111
visual_n <- 500
patterns <- c("cubic", "quartic", "sine", "two_moons")
sample_sizes <- c(100, 200, 300, 500, 750, 1000, 1500, 2000)
noise_levels <- seq(0, 1, length.out = 20)

# merge the simulation files
files <- list.files(
```

```
  here("experiments", "stringy_index", "results"),
  pattern = "\\.csv$",
  full.names = TRUE
)

all_results <- purrr::map_dfr(files, readr::read_csv, show_col_types = FALSE)

glimpse(all_results)
```

```
Rows: 64,000
Columns: 6
$ pattern    <chr> "cubic", "cubic", "cubic", "cubic", "cubic", "cubic", "cubi~
$ n          <dbl> 100, 100, 100, 100, 100, 100, 100, 100, 100, 100, 100, 100,~
$ noise_sd   <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,~
$ repetition <dbl> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, ~
$ seed       <dbl> 1112, 1113, 1114, 1115, 1116, 1117, 1118, 1119, 1120, 1121,~
$ stringy    <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,~
```

3.1 Summary of the results

For each pattern, sample size and noise level, we calculate the average Stringy05 score and how much it varies across the 100 repetitions. We also calculate the percentage of scores equal to zero.

```
summary_results <- all_results |>
  group_by(pattern, n, noise_sd) |>
  summarise(
    mean_stringy = mean(stringy),
    sd_stringy = sd(stringy),
    median_stringy = median(stringy),
    repetitions = n(),
    .groups = "drop"
  )

summary_results |>
  filter(n == visual_n) |>
  print(n = 12)
```

```
# A tibble: 80 x 7
   pattern     n noise_sd mean_stringy sd_stringy median_stringy repetitions
   <chr>   <dbl>    <dbl>        <dbl>      <dbl>          <dbl>       <int>
 1 cubic     500   0             1         0               1             100
 2 cubic     500   0.0526        0.920     0.0290          0.921         100
 3 cubic     500   0.105         0.696     0.0384          0.697         100
 4 cubic     500   0.158         0.524     0.0438          0.526         100
 5 cubic     500   0.211         0.418     0.0440          0.418         100
```

```
 6 cubic      500   0.263         0.342     0.0378         0.343         100
 7 cubic      500   0.316         0.286     0.0420         0.286         100
 8 cubic      500   0.368         0.245     0.0397         0.241         100
 9 cubic      500   0.421         0.209     0.0352         0.209         100
10 cubic      500   0.474         0.187     0.0397         0.182         100
11 cubic      500   0.526         0.156     0.0394         0.152         100
12 cubic      500   0.579         0.143     0.0383         0.141         100
# i 68 more rows
```

4 What do the original patterns look like?

```
make_structure <- function(pattern, n, seed = 1111) {
  set.seed(seed)

  if (pattern == "two_moons") {
    group <- sample(rep(1:2, length.out = n))
    theta <- runif(n, 0, pi)
    x <- ifelse(group == 1, cos(theta), 1 - cos(theta))
    y <- ifelse(group == 1, sin(theta), 0.5 - sin(theta))
  } else {
    t <- rnorm(n, 0, 1)
    x <- t

    if (pattern == "cubic")   y <- poly(t, degree = 3)[, 3]
    if (pattern == "quartic") y <- poly(t, degree = 4)[, 4]
    if (pattern == "sine")    y <- sin(pi * t)
  }

  tibble(
    x = as.numeric(scale(x)),
    y = as.numeric(scale(y))
  )
}

original_patterns <- purrr::map_dfr(patterns, function(p) {
  make_structure(p, visual_n, base_seed) |>
    mutate(pattern = p)
})
```

```
ggplot(original_patterns, aes(x, y)) +
  geom_point(size = 0.6, alpha = 0.7, colour = "#2679A8") +
  facet_wrap(~ pattern, ncol = 2) +
  coord_fixed() +
  labs(x = "Standardised X", y = "Standardised Y") +
  theme(aspect.ratio = 1)
```

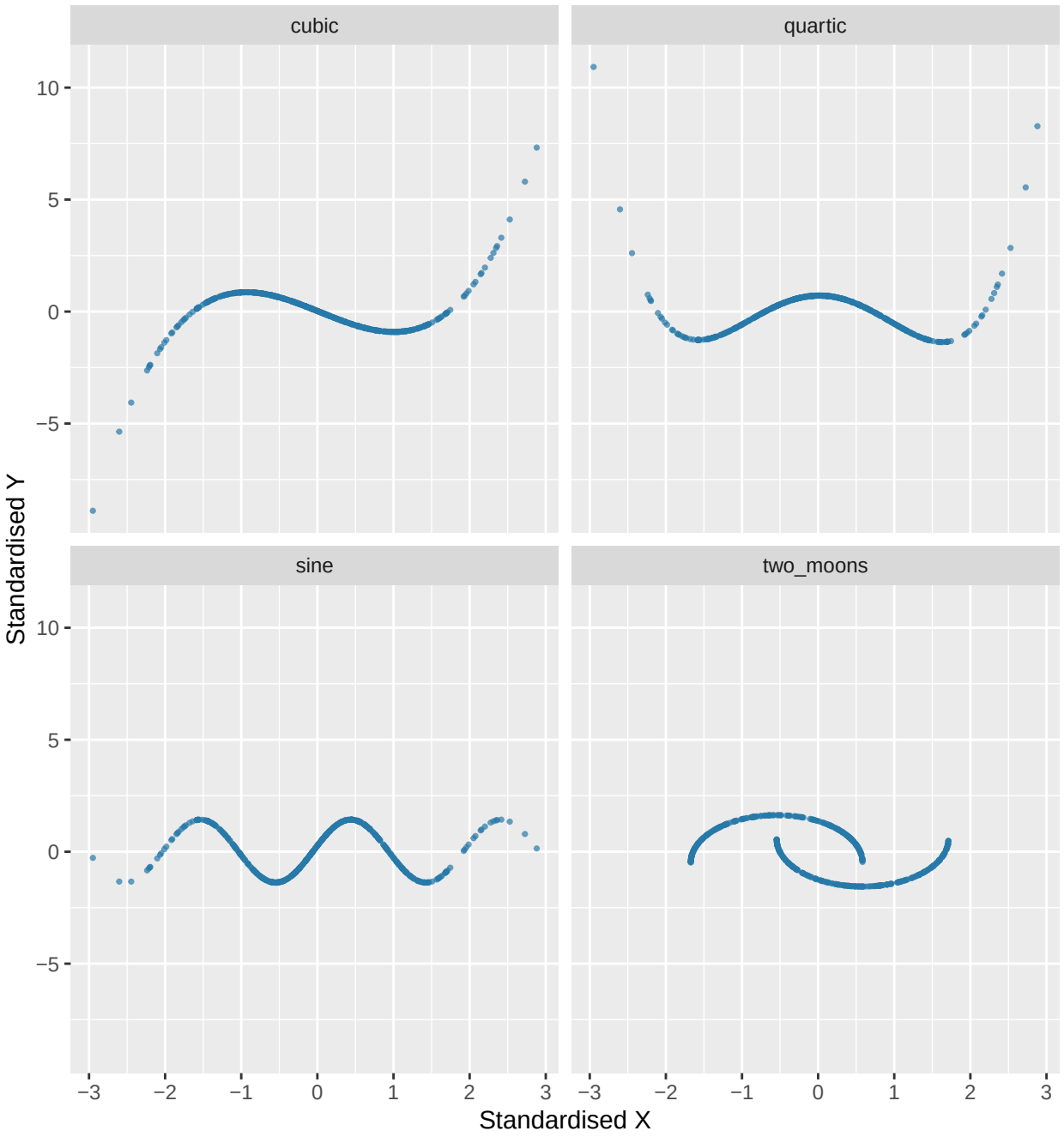


Figure 1: Original noise-free patterns at n = 500. Each coordinate has been standardised.

These patterns do not necessarily have the same noise-free Stringy05 score. Stringy05 captures one aspect of geometry; a visible pattern is not automatically highly Stringy.

```
baseline_scores <- summary_results |>
  filter(noise_sd == 0) |>
  select(pattern, n, baseline_stringy = mean_stringy)

knitr::kable(baseline_scores, digits = 3)
```

Table 1: Noise-free Stringy05 scores across patterns and sample sizes

pattern	n	baseline_stringy
cubic	100	1.000
cubic	200	1.000
cubic	300	1.000
cubic	500	1.000
cubic	750	1.000
cubic	1000	1.000
cubic	1500	1.000
cubic	2000	1.000
quartic	100	1.000
quartic	200	1.000
quartic	300	1.000
quartic	500	1.000
quartic	750	1.000
quartic	1000	1.000
quartic	1500	1.000
quartic	2000	1.000
sine	100	0.206
sine	200	0.643
sine	300	0.700
sine	500	0.653
sine	750	0.798
sine	1000	0.850
sine	1500	0.932
sine	2000	0.933
two_moons	100	0.822
two_moons	200	0.875
two_moons	300	0.832
two_moons	500	0.840
two_moons	750	0.842
two_moons	1000	0.845
two_moons	1500	0.851
two_moons	2000	0.852

5 How does each pattern change as noise increases?

We first focus on sample size **500**, since this is the intended size of the final benchmark datasets. We show six noise levels spanning the experiment, including no noise and SD = 1.

At each level, we select the repetition whose Stringy05 score is closest to the **median score** across repetitions. We then recreate its X/Y coordinates using its saved seed. The label reports the **mean Stringy05 over all 100 repetitions**, not the score of the pictured repetition.

```r
# Choose six positions from the 20-level noise grid
display_sd <- noise_levels[c(1, 2, 4, 7, 12, 20)]

selected_runs <- all_results |>
  filter(n == visual_n) |>
  filter(round(noise_sd * 19) %in% round(display_sd * 19)) |>
  group_by(pattern, noise_sd) |>
  mutate(distance = abs(stringy - median(stringy))) |>
  slice_min(distance, n = 1, with_ties = FALSE) |>
  ungroup() |>
  left_join(
    summary_results |>
      filter(n == visual_n) |>
      select(pattern, noise_sd, mean_stringy),
    by = c("pattern", "noise_sd")
  )

# Recreate only the small number of datasets needed for the figure
plot_data <- purrr::pmap_dfr(
  selected_runs,
  function(pattern, n, noise_sd, repetition, seed, stringy,
           distance, mean_stringy, ...) {
    original <- make_structure(pattern, n, base_seed)
    set.seed(seed)
    noise_x <- rnorm(n, 0, noise_sd)
    noise_y <- rnorm(n, 0, noise_sd)

    tibble(
      pattern = pattern,
      noise_sd = noise_sd,
      mean_stringy = mean_stringy,
      x = original$x + noise_x,
      y = original$y + noise_y
    )
  }
)

# Data for printing mean values in the panels
panel_means <- plot_data |>
  distinct(pattern, noise_sd, mean_stringy)
```

```r
# Use the same plot boundaries for every scatterplot
x_min <- min(plot_data$x)
y_max <- max(plot_data$y) + 0.8

plot_data <- plot_data |>
  mutate(noise_label = factor(
```

```
    sprintf("SD = %.3f", noise_sd),
    levels = sprintf("SD = %.3f", display_sd)
  ))

panel_means <- panel_means |>
  mutate(noise_label = factor(
    sprintf("SD = %.3f", noise_sd),
    levels = sprintf("SD = %.3f", display_sd)
  ))

ggplot(plot_data, aes(x, y)) +
  geom_point(size = 0.45, alpha = 0.65, colour = "#2679A8") +
  geom_text(
    data = panel_means,
    aes(x = x_min, y = y_max,
        label = sprintf("Mean = %.3f", mean_stringy)),
    inherit.aes = FALSE,
    hjust = 0, size = 3
  ) +
  facet_grid(pattern ~ noise_label) +
  coord_fixed(
    xlim = range(plot_data$x) + c(-0.3, 0.3),
    ylim = c(min(plot_data$y) - 0.3, y_max + 0.3)
  ) +
  labs(x = "Noisy X", y = "Noisy Y") +
  theme(aspect.ratio = 1)
```

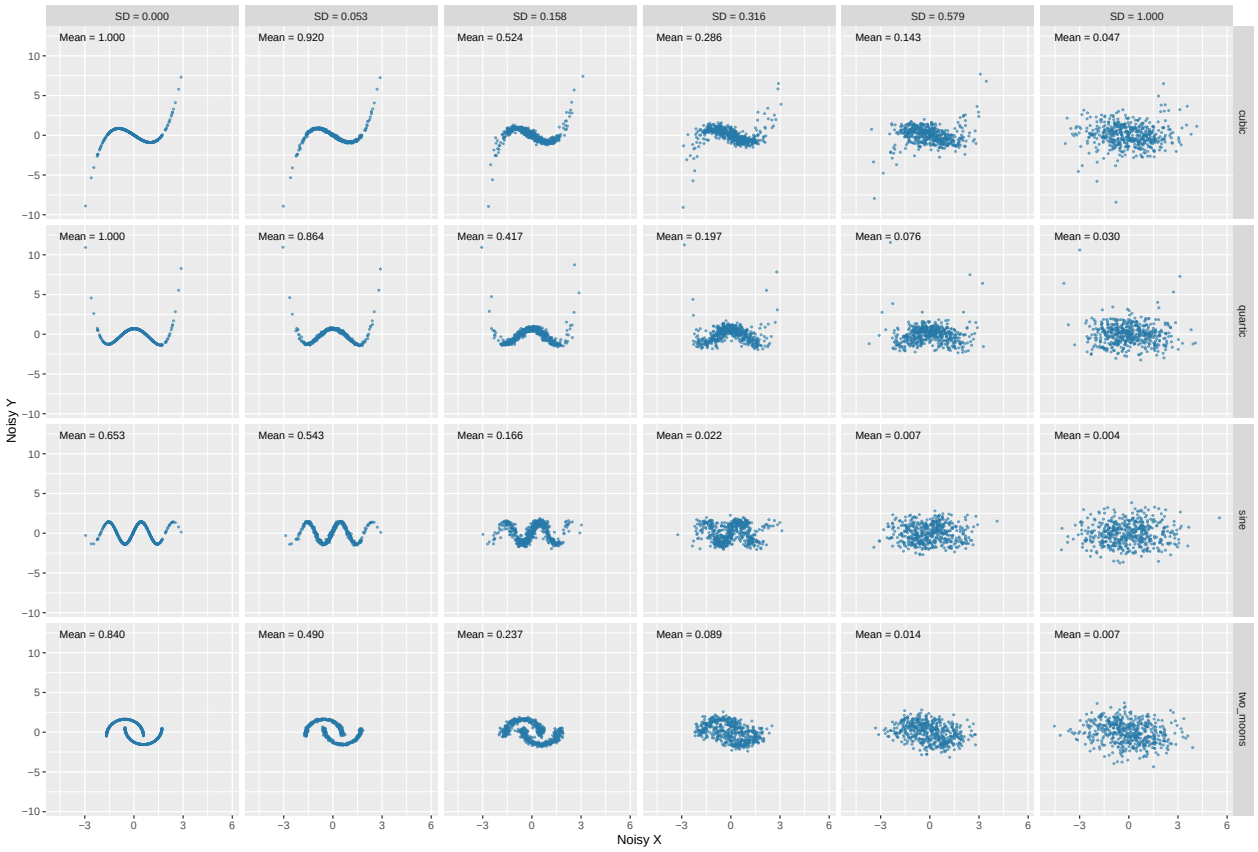


Figure 2: Selected noise levels for four patterns at n = 500. Each panel shows one representative realisation; the printed Stringy05 is the mean across repetitions.

These plots show how the point clouds change as noise grows. Importantly, a pattern may remain visually recognisable even if Stringy05 becomes small or reaches zero.

6 How does Stringy05 change with noise?

6.1 Mean Stringy05 for the four patterns at n = 500

The following figure shows the average index score at each noise level. A faster decline indicates that the *measured Stringy property* is more sensitive to noise under that pattern and sample size.

```
ggplot() +
  # Individual Stringy05 scores from every simulation
  geom_point(
    data = all_results |> filter(n == visual_n),
    aes(x = noise_sd, y = stringy),
    colour = "steelblue",
    alpha = 0.3,
    size = 0.7
```

```
) +
# Mean Stringy05 at each noise level
geom_line(
  data = summary_results |> filter(n == visual_n),
  aes(x = noise_sd, y = mean_stringy),
  colour = "darkgreen",
  linewidth = 1.2
) +
facet_wrap(~ pattern, ncol = 2) +
coord_cartesian(ylim = c(0, 1)) +
labs(
  x = "Gaussian noise SD",
  y = "Rescaled Stringy05"
) +
theme(aspect.ratio = 1)
```

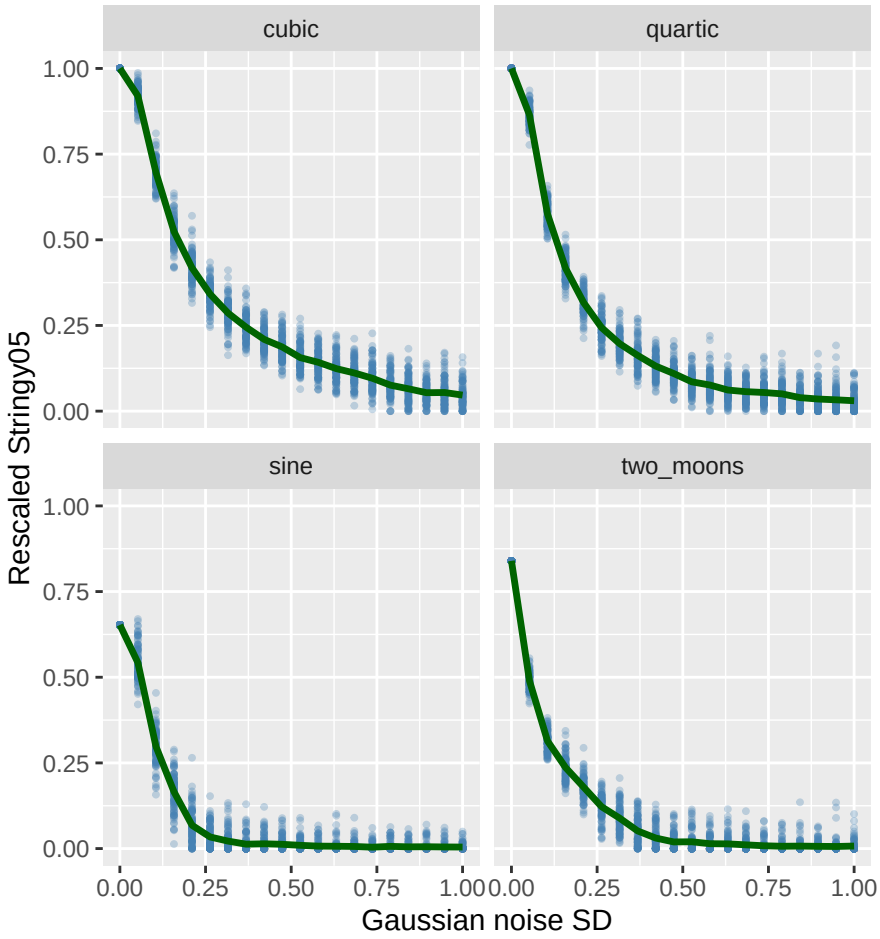


Figure 3: Mean rescaled Stringy05 versus Gaussian noise SD for four patterns at n = 500.

6.2 Variation across repetitions

Mean scores do not tell us how consistent the index is. Boxplots show the spread of the 100 Stringy05 values for each noise level.

```
all_results |>
  filter(n == visual_n) |>
  ggplot(aes(x = factor(round(noise_sd, 3)), y = stringy)) +
  geom_boxplot(fill = "#8abfdb", outlier.alpha = 0.4) +
  facet_wrap(~ pattern, ncol = 2) +
  labs(x = "Noise SD", y = "Rescaled Stringy05") +
  theme(aspect.ratio = 1,
        axis.text.x = element_text(angle = 55, hjust = 1))
```

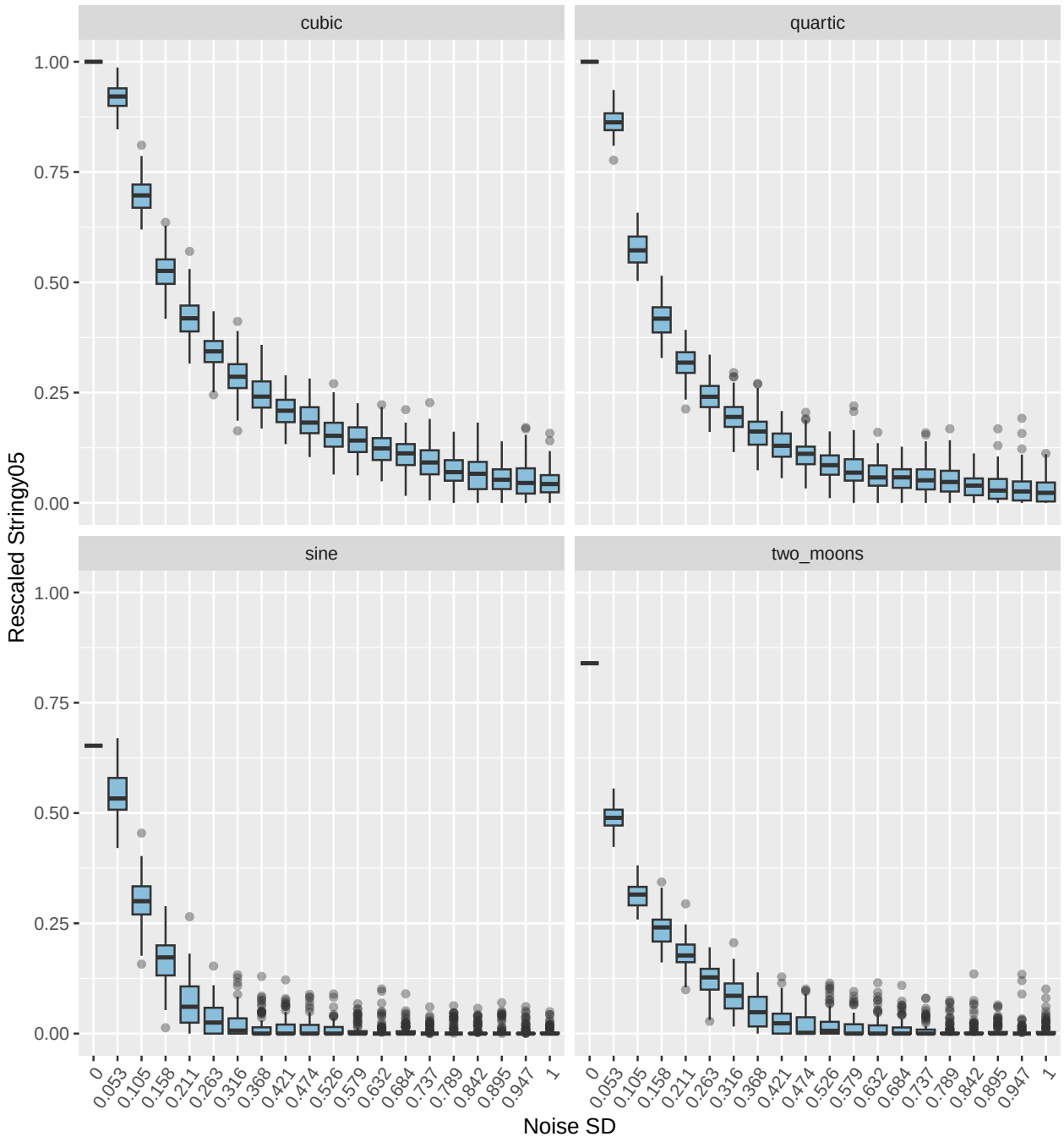


Figure 4: Stringy05 distributions across repetitions for each pattern at n = 500.

7 Does sample size change the response to noise?

We also simulated eight sample sizes. The next figures show whether smaller and larger samples produce different Stringy05 responses under the same noise levels.

7.1 Noise-response curves for every sample size

```
summary_results |>
  ggplot(aes(noise_sd, mean_stringy, colour = pattern)) +
  geom_line(linewidth = 0.9) +
  facet_wrap(~ n, ncol = 4) +
  coord_cartesian(ylim = c(0, 1)) +
  labs(x = "Noise SD", y = "Mean Stringy05", colour = "Pattern") +
  theme(aspect.ratio = 1)
```

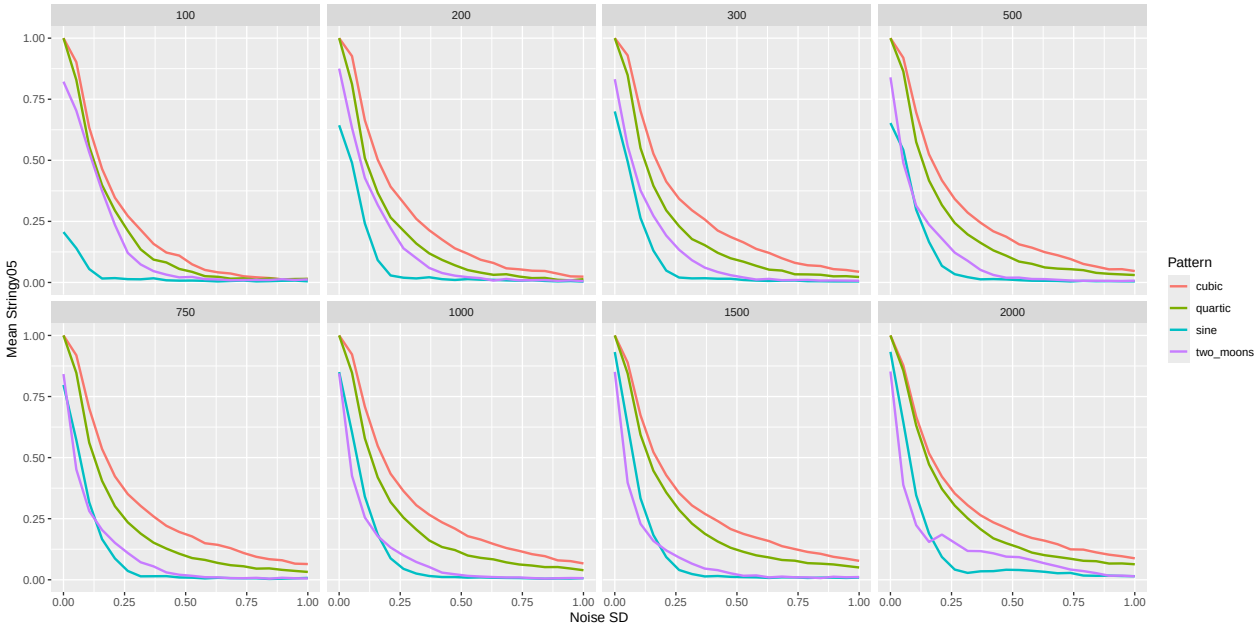


Figure 5: Mean Stringy05 against noise SD for each pattern, separated by sample size.

7.1.1 Noise sensitivity and variability across sample sizes

To investigate how the rescaled Stringy05 index responds to increasing Gaussian noise, we compare its distribution across all four geometric patterns and eight sample sizes. Each panel shows the individual Stringy05 scores from 100 repetitions at each noise level, together with the mean score (solid line) and the 95% empirical interval (shaded region). This allows us to examine not only how the average Stringy05 changes as noise increases, but also how its variability depends on the underlying pattern and sample size.

```
# Calculate the distribution of scores
plot_summary <- all_results |>
  group_by(pattern, n, noise_sd) |>
  summarise(
    mean_stringy = mean(stringy),
    lower = quantile(stringy, 0.025),
```

```
    upper = quantile(stringy, 0.975),
    .groups = "drop"
  )

ggplot() +
  # 95% range of observed scores
  geom_ribbon(
    data = plot_summary,
    aes(
      x = noise_sd,
      ymin = lower,
      ymax = upper,
      fill = pattern,
      group = pattern
    ),
    alpha = 0.15
  ) +

  # All 100 repetitions
  geom_point(
    data = all_results,
    aes(noise_sd, stringy, colour = pattern),
    alpha = 0.12,
    size = 0.35,
    position = position_jitter(width = 0.004, height = 0)
  ) +
  # Mean Stringy05
  geom_line(
    data = plot_summary,
    aes(noise_sd, mean_stringy, colour = pattern),
    linewidth = 0.9
  ) +

  facet_grid(pattern ~ n) +
  coord_cartesian(ylim = c(0, 1)) +
  labs(
    x = "Gaussian noise SD",
    y = "Rescaled Stringy05",
    colour = "Pattern",
    fill = "Pattern"
  )  +
  theme(aspect.ratio = 1,
    strip.text = element_text(face = "bold"),
    legend.position = "none"
  )
```

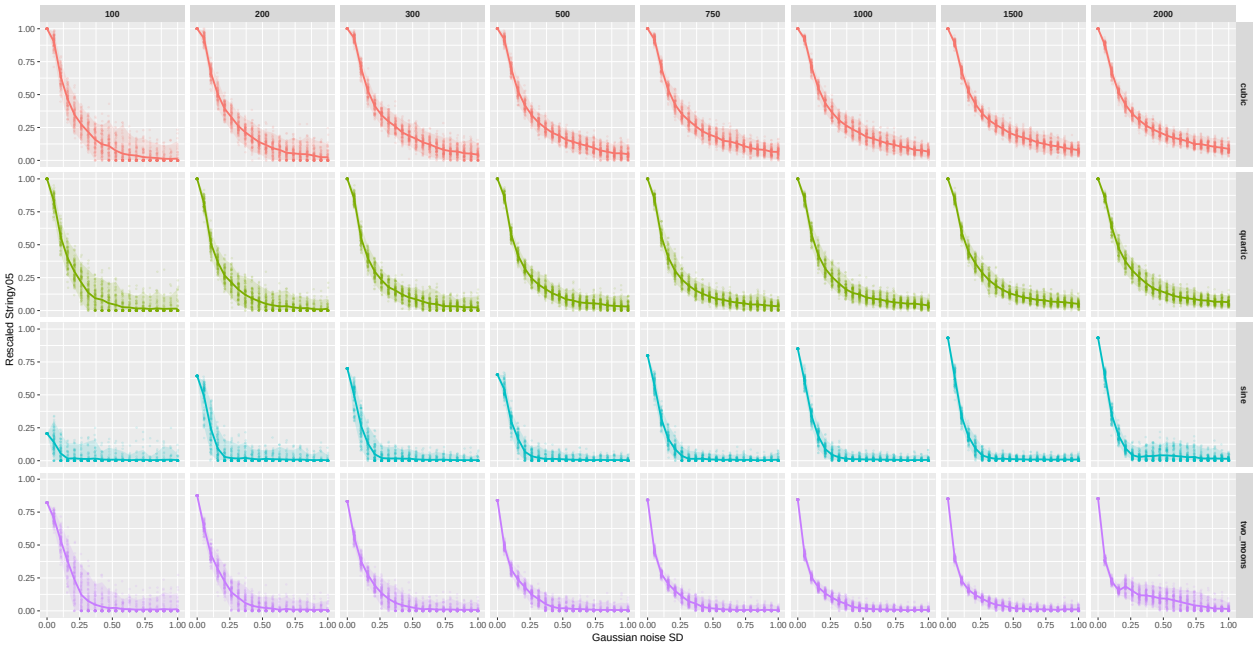


Figure 6: Noise sensitivity of Stringy05 across patterns and sample sizes. Points show individual repetitions, lines show means, and shaded bands show the 95% empirical range.

7.2 Heatmap of pattern, sample size and noise

Each square shows the mean Stringy05 score for a sample size and noise level. This makes it easier to see whether changes are gradual or occur within a narrow range of noise SDs.

```
summary_results |>
  ggplot(aes(
    x = factor(round(noise_sd, 3)),
    y = factor(n),
    fill = mean_stringy
  )) +
  geom_tile() +
  facet_wrap(~ pattern, ncol = 2) +
  scale_fill_viridis_c() +
  labs(
    x = "Gaussian noise SD",
    y = "Sample size",
    fill = "Mean Stringy05"
  ) +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 55, hjust = 1))
```

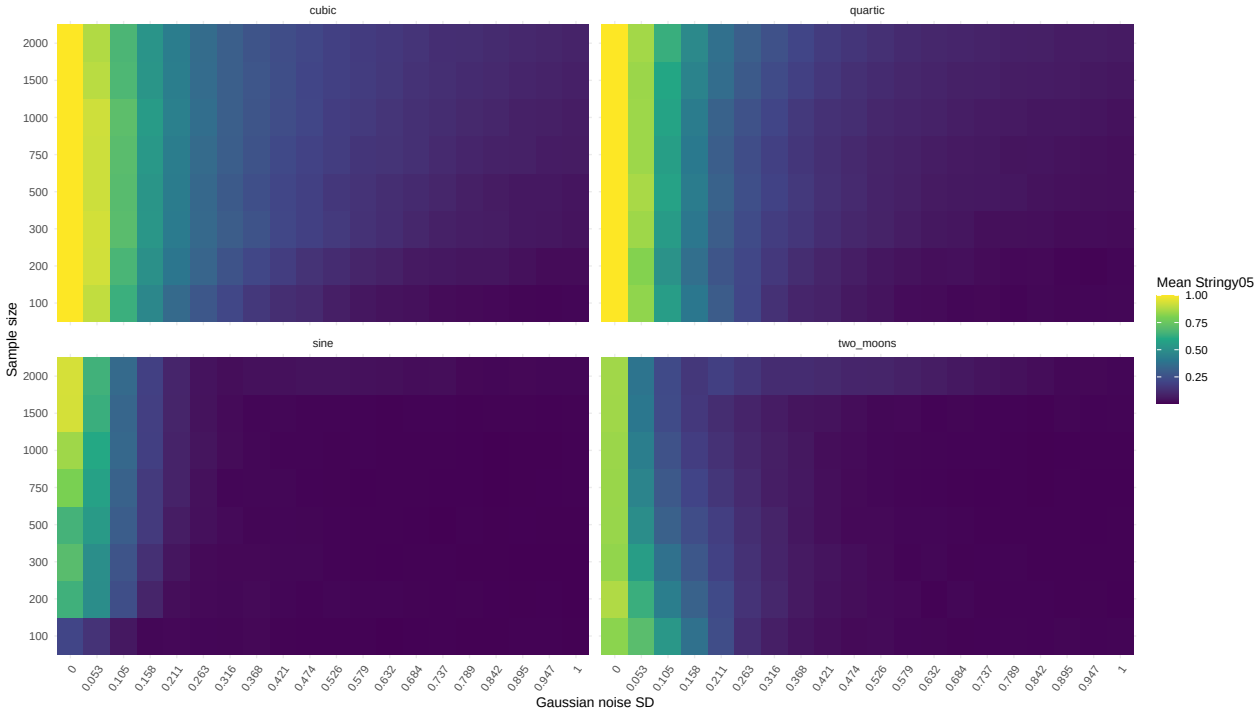


Figure 7: Mean rescaled Stringy05 by noise SD and sample size, for each geometric pattern.

This study investigated the sensitivity of the rescaled Stringy05 index to Gaussian noise across four geometric patterns and eight sample sizes. The results indicate that Stringy05 generally decreases as noise increases, but its robustness varies considerably across patterns. Cubic and quartic structures retain higher index values under moderate noise, whereas sine and two-moons structures are more sensitive to perturbation. Larger sample sizes generally improve the persistence of the index, although the effect is pattern-dependent.

These findings highlight the importance of considering both geometric structure and sample size when evaluating the noise sensitivity of Stringy05. For future projection pursuit experiments, noise levels should be selected separately for each pattern rather than applying a single noise threshold to all structures.

The next stage will focus on selecting suitable noise levels for synthetic datasets with 500 observations, using the observed Stringy05 responses to construct examples with different levels of difficulty.

7.3 Interpretation of results

The simulation results show that the rescaled Stringy05 index is sensitive to increasing Gaussian noise, although the rate of decline differs across geometric patterns and sample sizes. Cubic and quartic patterns generally retain higher Stringy05 scores over a wider range of noise levels, while sine and two-moons patterns show a more rapid decrease. This suggests that the noise sensitivity of Stringy05 depends on the underlying geometric structure.

Sample size also influences the behaviour of the index. In general, larger samples retain higher Stringy05 scores under the same noise conditions, particularly for cubic and quartic patterns. However, this relationship is not consistent across all patterns and noise levels. The distributions

across repetitions also show that noise introduces variability in the index, with scores generally approaching zero as noise increases.

The heatmaps further illustrate how Stringy05 changes jointly with sample size and noise level. They show that the transition from high to low Stringy05 scores occurs at different noise levels depending on the pattern. Therefore, the same amount of Gaussian noise does not necessarily produce the same reduction in Stringy05 across different geometric structures.

Overall, these findings highlight the importance of considering both geometric structure and sample size when evaluating Stringy05. Based on the visualisations and observed index responses, noise standard deviations between 0 and 0.3 appear to provide a useful initial range for investigating structural degradation, although the appropriate noise level varies across patterns.

These results will inform the next stage of this research, which focuses on generating synthetic high-dimensional datasets with hidden geometric structures for evaluating projection pursuit methods. The aim is to investigate whether projection pursuit can recover meaningful structures when they are mixed with nuisance variables. The dataset-generation procedure and its evaluation will be discussed separately.